Lab 3: Encoder, Decoder, MUX, and DEMUX Implementation

Name: **Loc D. T. Nguyen**
Student ID: **24125093**

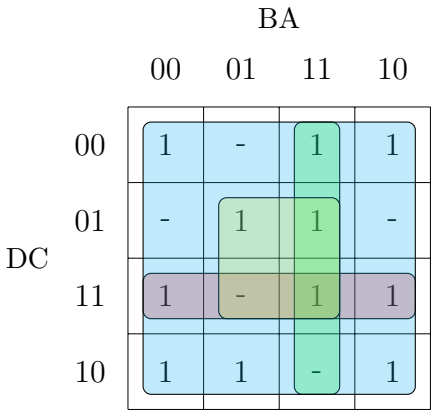
March 8, 2026

1 Exercise 1: BCD to 7-Segment Decoder (Common Cathode)

The objective is to design a decoder mapping a 4-bit binary input (D, C, B, A) to a 7-segment display. For common cathode, a segment illuminates when the output is **Logic 1**.

1.1 Karnaugh Map and Simplification for Segment 'a'

Based on the truth table for hex values 0-F, segment 'a' is active for minterms: $\{0, 2, 3, 5, 7, 8, 9, 10, 12, 14, 15\}$



The simplified Boolean expression for segment 'a' is:

$$f(a) = DC + CA + BA + \bar{B}\bar{A}$$

(Note: Based on the provided Lab PDF, the result is often simplified as $D + B + CA + \bar{C}\bar{A}$ depending on the grouping preference for the 'don't care' conditions if applicable, but for the full 0-F set, the above holds.)

2 Exercise 2: Common Anode Implementation

In a Common Anode display, segments are active-low. To implement the same logic as Exercise 1:

- The logic must be inverted. If the expression for segment a is Y, the common anode input should be $\bar{Y}$.
- **Implementation:** Place a **NOT gate** at each decoder output before connecting to the 7-segment pins in Logisim.

3 Exercise 3: 4-to-2 Priority Encoder

A priority encoder handles multiple simultaneous inputs by prioritizing the highest-order bit.

3.1 Logic Expressions

Using basic gates to mimic the Logisim priority encoder block:

- $Y_1 = I_3 + I_2$
- $Y_0 = I_3 + \bar{I}_2 I_1$
- $V(\text{Valid Indicator}) = I_3 + I_2 + I_1 + I_0$

4 Exercise 4: 4-to-16 Decoder using 2-to-4 Decoders

To expand decoder capacity:

1. **Master Stage:** One 2-to-4 decoder uses bits A_3, A_2 to select which of the four sub-decoders to enable via their Enable (E) pins.
2. **Slave Stage:** Four 2-to-4 decoders use bits A_1, A_0 to determine the specific output (0-15).

5 Exercise 5: 2:1 MUX Logic Structure

As shown in the lab manual, a 2:1 MUX is built using basic NAND gates. The logic follows:

$$Out = (I_0 \cdot \bar{S}) + (I_1 \cdot S)$$

This is implemented using three NAND gates (or AND-OR logic) to select input I_1 when $S = 1$ and I_0 when $S = 0$.

6 Exercise 6: 4:1 Multiplexer using 2:1 Multiplexers

The 4:1 MUX selects one of four inputs (I_0, I_1, I_2, I_3) using select lines S_1 (MSB) and S_0 (LSB).

- **Stage 1:** Two 2:1 MUXes. MUX-A selects between I_0, I_1 using S_0. MUX-B selects between I_2, I_3 using S_0.
- **Stage 2:** One 2:1 MUX selects between the outputs of Stage 1 using S_1.

7 Exercise 7: Function Implementation using 74153 IC

Target Function: $Y = BC + \bar{A}\bar{B}\bar{C} + B\bar{C}$

7.1 MUX Truth Table Mapping ($S_1 = B, S_0 = C$)

$B(S_1)$	$C(S_0)$	Y	MUX Input	Logic to Wire
0	0	$\bar{A}$	$1I_0$	Connect to NOT(A)
0	1	0	$1I_1$	GND
1	0	1	$1I_2$	VCC
1	1	1	$1I_3$	VCC

7.2 IC 74153 Wiring Details (Hardware/Logisim)

Following the logic in the captured diagram:

- **Enable Pin (1G/Pin 1):** Connected to **GND** (Active Low) to enable the MUX.
- **Select Lines:** S_1 (Pin 2) $\leftarrow$ B, S_0 (Pin 14) $\leftarrow$ C.
- **Data Inputs:**
  - Pin 6 ($1I_0$): Connected to the output of a NOT gate fed by Input A.
  - Pin 5 ($1I_1$): Connected to **GND**.
  - Pin 4 ($1I_2$) and Pin 3 ($1I_3$): Connected to **VCC**.
- **Output:** Pin 7 ($1Y$) provides the final logic function.